UAV-RT pulseplotter

Overview and user guide

Dynamic and Active Systems Lab, Northern Arizona University

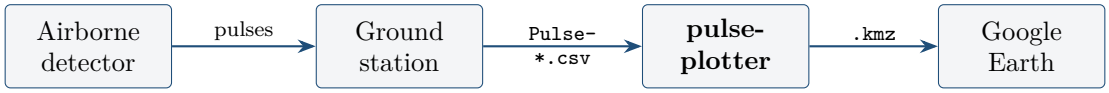
September 2, 2026

In one sentence. pulseplotter reads the pulse log recorded during a radio-telemetry flight, maps received signal strength over the area you flew, estimates a bearing to the tag, and writes a KMZ you can open in Google Earth.

Contents

1 Where it sits

pulseplotter is the post-flight end of the UAV-RT system. During a flight the airborne detector finds tag pulses and the ground station records them; this tool picks up after the aircraft lands.



The airborne detector is `uavrt_detection` or `MavlinkTagController2`; the ground station is `TagTracker`. This tool does four things:

1. Reads any of the four pulse-log formats TagTracker has shipped.
2. Plots each pulse at the position the aircraft held when it was received, coloured by a property you choose, over an interpolated surface.
3. Estimates a bearing from the aircraft to the tag, with a confidence figure.
4. Exports the whole thing as a self-contained KMZ.

There are two implementations. They are the same tool: same inputs, same outputs, same numbers. Use whichever suits you.

	`matlab/`	`python/`
Run it	`clear classes; pulseplotter2`	`python pulseplotter.py`
Needs	MATLAB, base install, no toolboxes	Python 3.8+, numpy, matplotlib
Licence cost	MATLAB licence	none

2 Getting started

2.1 Python, no MATLAB licence needed

You need Python 3.8 or newer *with Tk 8.6 or newer*. Check first, because Apple's system Tk is 8.5.9 and the window will come up almost empty with no error:

```
python3 -c "import tkinter; print(tkinter.TkVersion)"
```

If that prints `8.5`, install a Python that bundles Tk 8.6 (`brew install python@3.12 python-tk@3.12`, or the python.org installer) and build the environment from it *by full path*. Then:

```
cd python
python3 -m venv .venv && source .venv/bin/activate
python -m pip install -r requirements.txt
python pulseplotter.py
```

On Windows the middle two lines are `py -m venv .venv` and `.venv\Scripts\activate`. Every new terminal needs the activate step again. You can pass a log on the command line to open it straight away.

2.2 MATLAB

```
cd matlab
clear classes      % MATLAB caches classdefs — do not skip this after an edit
pulseplotter2
```

Run `pulseplotter2`, not the `.mlapp`. The four helper files (`readpulsetable.m`, `geo2enu.m`, `enu2geo.m`, `kmzwrite.m`) must sit in the same folder — the app calls them by name.

3 The pulse log

One file, one flight. Each row is one detected pulse, carrying the tag it belongs to, when it arrived, how strong it was, and where the aircraft was.

TagTracker has shipped four header variants of this file, including a 2023 build with no header at all. In every one of them the first sixteen fields are in the same order and columns 14, 15 and 16 are latitude, longitude and altitude. The reader therefore parses *positionally* and ignores what the header calls things, so every variant loads without you having to know which build wrote it.

Two things it handles that a naive CSV reader does not:

- The leading `#` on the header line. You do not need to delete it.
- The four-field rotation start/stop records interleaved among the pulses. Read naively these become bogus "pulses" carrying a latitude in the `tag_id` column, which corrupts the tag list and the plot origin.

The two logs used throughout this guide are real flights from the November 2025 Cumbria campaign:

Flight	File	Pulses	Duration	Tags
Day 5, 21 Nov	`Pulse-2025-11-21-11-08-41-149.csv`	1111	13.3 min	40, 41, 42, 43
Day 8, 24 Nov	`Pulse-2025-11-24-15-42-40-343.csv`	460	10.5 min	10, 12, 13

4 A worked example

4.1 Load a flight

Press **Load Data** and choose `Pulse-2025-11-21-11-08-41-149.csv`. The plot appears immediately — there is no separate "plot" button. The **Tag ID** list fills with the tags present in the file and selects the most common one.

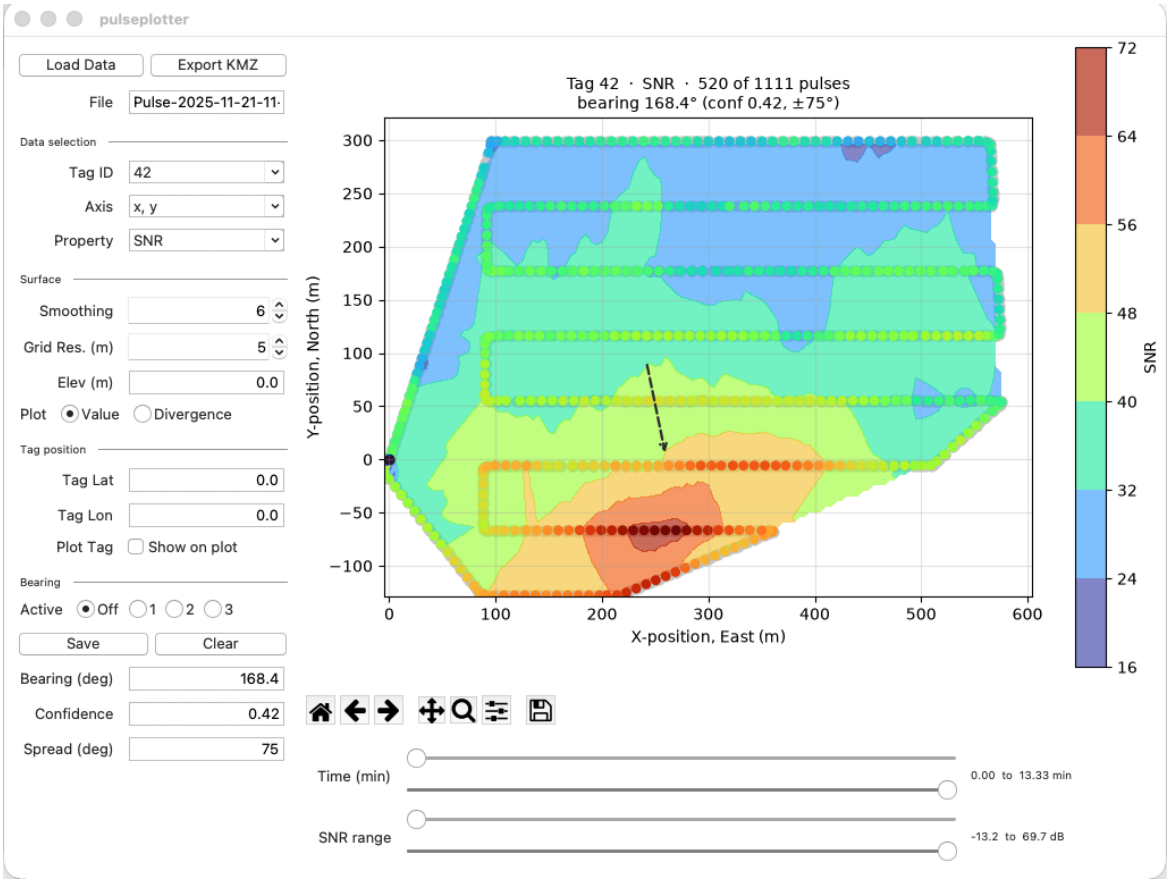


Figure 1: The application window, Day 5 loaded and tag 42 selected. The control column is on the left in four groups; the plot, the toolbar and the two range sliders each get their own strip on the right. The dashed arrow is the current bearing estimate, and the same three numbers appear under **Bearing** in the control column and in the plot title. Both implementations use this arrangement and these four groups.

4.2 Pick a tag

This flight carried four tags. Changing **Tag ID** replots for that tag alone; the grey dots stay put and show every pulse in the file, so you can see how much of the flight the selected tag actually covers.

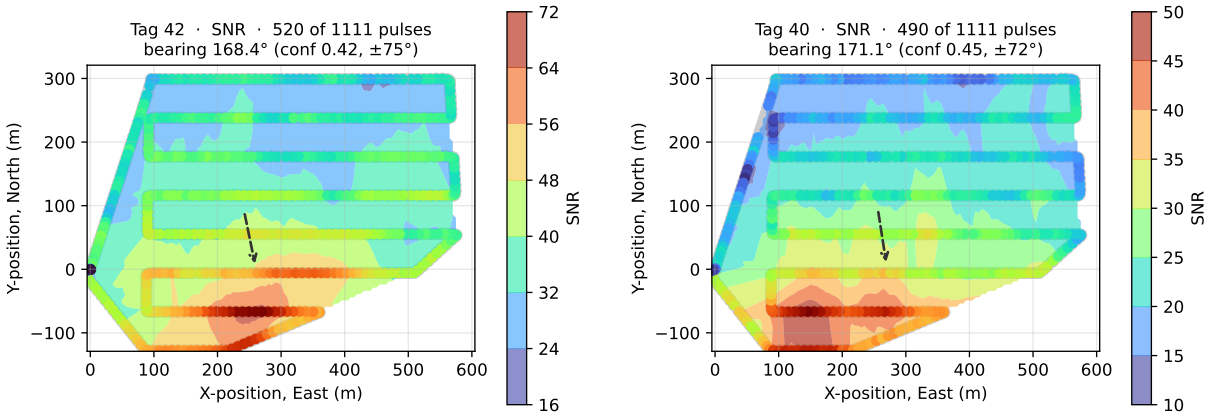


Figure 2: Day 5, tag 42 (left, 520 pulses) and tag 40 (right, 490 pulses) from the same flight. Grey dots are all 1111 pulses; coloured dots are the selected tag; the filled surface is SNR interpolated over the area flown. Both tags put the strong signal to the south-east and give bearings within three degrees of each other. Real output from the application.

4.3 Narrow the selection

The two range sliders under the plot cut the data down by time and by SNR. Each is a lower and an upper bound; the plot redraws when you release the handle, not while you drag. Use the time slider to isolate one leg of the survey, and the SNR slider to drop noise-floor detections that would otherwise flatten the surface.

4.4 Choose what to plot

Property selects what colours the dots and drives the surface: SNR, STFT score, time, or altitude. **Axis** switches between local metres and degrees. **Plot Property** switches the surface between the interpolated value and its divergence, which highlights where the field converges.

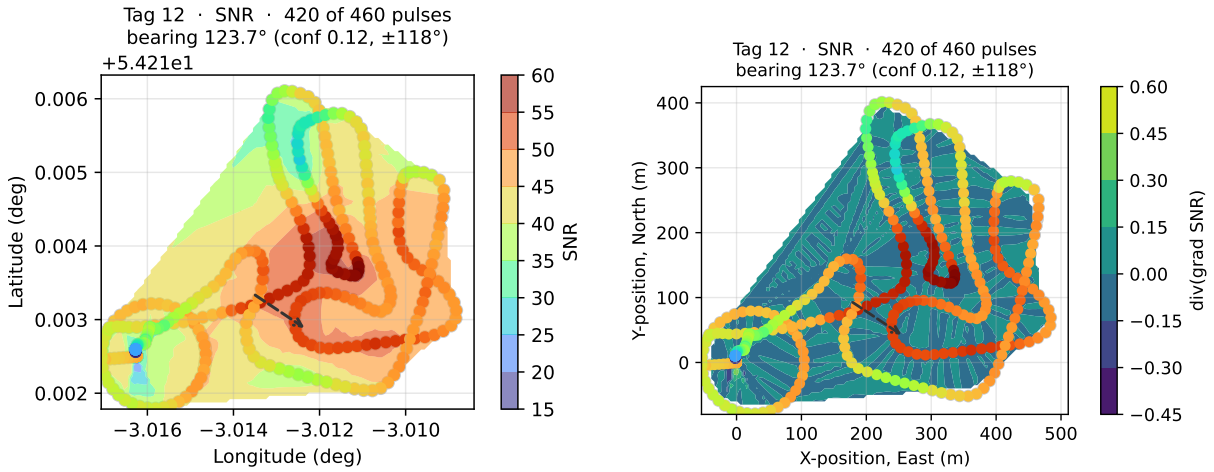


Figure 3: Day 8, tag 12. Left: the same data in `Lon, Lat` rather than `x, y`; the aspect ratio is corrected for latitude so the map is not stretched east–west. Right: **Divergence** mode, which shows where the gradient field converges — the dark region is where the surface peaks.

4.5 Export

Export KMZ (**Export KML** in MATLAB) writes a single self-contained archive. Set **Elev (m)** to the takeoff elevation in metres MSL first, if you want the pulse markers at their true absolute

altitude.

5 Controls

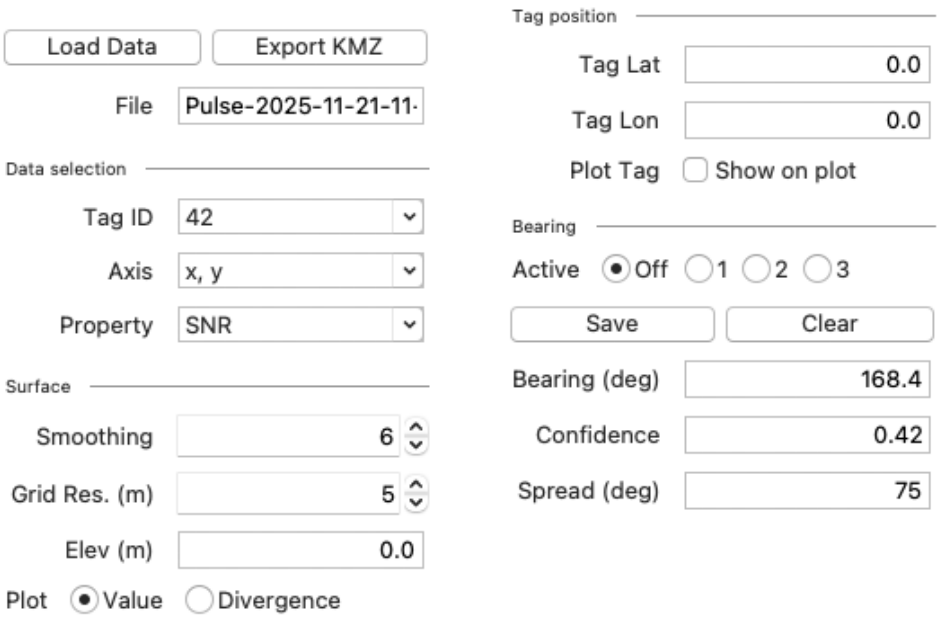


Figure 4: The control column, shown as two halves side by side so it fits the page. Values are from the Day 5 flight. The four groups are **Data selection**, **Surface**, **Tag position** and **Bearing**, in that order down the column.

Data selection	
Tag ID	Which tag to analyse. Defaults to the most common one in the file.
Axis	Local metres (`x, y`) or degrees (`Lon, Lat`).
Property	What to colour and contour: SNR, STFT Score, Time, Altitude (m).
Start/End Time	The current time window, in seconds from the first pulse. Read-only; driven by the time slider.
Surface	
Smoothing	Moving-mean window over the property, in pulses. 0 or 1 disables it.
Grid Res. (m)	Interpolation grid spacing. Coarsened automatically if a fine grid over a long flight would be too slow.
Elev (m)	Takeoff elevation, metres MSL. Added to the relative pulse altitudes when writing KML.
Plot	Show the interpolated value, or its divergence.
Tag position	
Tag Lat / Lon	A known tag position, for comparison.
Plot Tag	Draw it on the plot and include it in the export.
Bearing	
Active	Which of the three save slots is live; **Off** disables saving.
Save / Clear	Store the current bearing in that slot, or empty it. Saved bearings are drawn as solid arrows, the current one as a dashed arrow, so you can compare segments of a flight.
Bearing, Confidence, Spread	The current estimate. See section 6.
Below the plot	
Time / SNR sliders	Lower and upper bounds. Replot on release.

Loading a new file clears all three saved bearing slots: they belong to the flight they were measured on.

6 How the bearing is computed

6.1 The idea

Received signal strength falls off with distance from the transmitter. If that is the dominant effect over the area you flew, then the interpolated SNR surface slopes upward towards the tag, and its gradient points at it. The estimator takes the gradient everywhere on the grid and asks what direction those arrows agree on.

6.2 Local frame

Pulse positions arrive as latitude, longitude and relative altitude. They are converted to a local east–north–up frame anchored on the first pulse of the flight, (φ_0, λ_0), using the WGS84 meridional and normal radii of curvature evaluated at that latitude:

$$R_M = \frac{a(1-e^2)}{(1-e^2\sin^2\varphi_0)^{3/2}}, \qquad R_N = \frac{a}{\sqrt{1-e^2\sin^2\varphi_0}}, \tag{1}$$

$$x_i = (\lambda_i - \lambda_0)\,\frac{\pi}{180}\,R_N\cos\varphi_0, \qquad y_i = (\varphi_i - \varphi_0)\,\frac{\pi}{180}\,R_M. \tag{2}$$

This flat-earth approximation agrees with a rigorous ECEF-based transform to better than 5 cm over a 1 km box. Because it is linear in x and y independently, a rectangular grid in this frame maps to an exactly rectangular latitude/longitude box — which is what lets the KML ground overlay be exact rather than approximate.

6.3 Surface

Let P_i be the chosen property at pulse i, optionally replaced by a centred moving mean over w consecutive pulses. The scattered values are interpolated onto a regular grid of spacing Δ by linear interpolation over a Delaunay triangulation, giving $\hat{P}_k$ at cell k. Cells outside the convex hull of the flown positions are left undefined and take no part in anything that follows — this is also why the exported overlay is transparent there.

6.4 Resultant

Central differences give the gradient $\nabla\hat{P}_k = (F_{x,k}, F_{y,k})$ at each cell. Let $\mathcal{K}$ be the cells where both components are finite and the gradient is non-zero. The estimator forms the magnitude-weighted circular mean of the gradient directions: with unit directions $\hat{u}_k = \nabla\hat{P}_k/\|\nabla\hat{P}_k\|$ and weights $w_k = \|\nabla\hat{P}_k\|$,

$$\mathbf{R} \;=\; \sum_{k\in\mathcal{K}} w_k\hat{u}_k \;=\; \sum_{k\in\mathcal{K}} \nabla\hat{P}_k, \qquad W \;=\; \sum_{k\in\mathcal{K}} \|\nabla\hat{P}_k\|. \tag{3}$$

The direction of $\mathbf{R}$ is reported as a compass bearing — zero at north, increasing clockwise — and its normalised length as a confidence:

$$\theta = \operatorname{atan2}(R_y, R_x), \qquad \beta = (90^\circ - \theta) \bmod 360^\circ, \tag{4}$$

$$R = \min\!\left(\frac{\|\mathbf{R}\|}{W},\, 1\right), \qquad \sigma = \sqrt{-2\ln R} \quad \text{(radians)}. \tag{5}$$

σ is the circular standard deviation implied by a resultant length R, and is the ± figure in the plot title.

What the weighting does, and does not, do. The two forms in equation (3) are equal, because the weight w_k cancels the normalisation in $\hat{u}_k$. The reported *direction* is therefore exactly the direction of the plain vector sum of the gradient field, identical to what averaging the components would give. What the circular framing contributes is R and σ: the ratio of the length of the summed vector to the sum of the lengths, which is a genuine measure of how much the field agrees and has no counterpart in a component average.

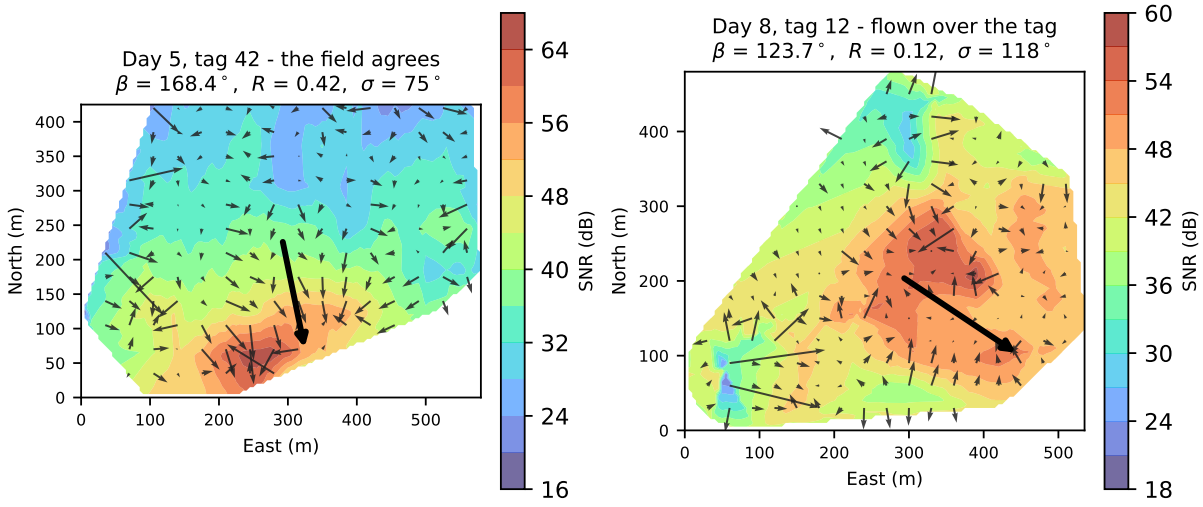


Figure 5: The gradient field the estimate is built from, on two real flights. Left: the arrows broadly agree and the resultant is meaningful. Right: the aircraft passed over the tag, so the gradients converge on an interior maximum and cancel; the resultant still has a direction but $R = 0.12$ says not to trust it. Both panels are computed by the application's own code.

6.5 Reading the confidence

R runs from 0 to 1 and measures how much the cells of the field agree on a direction, not how far the answer is from the truth. It is sensitive: a small amount of scatter on the surface pulls it down sharply while the bearing itself is still good, because independent noise in the gradient largely cancels when summed over thousands of cells.

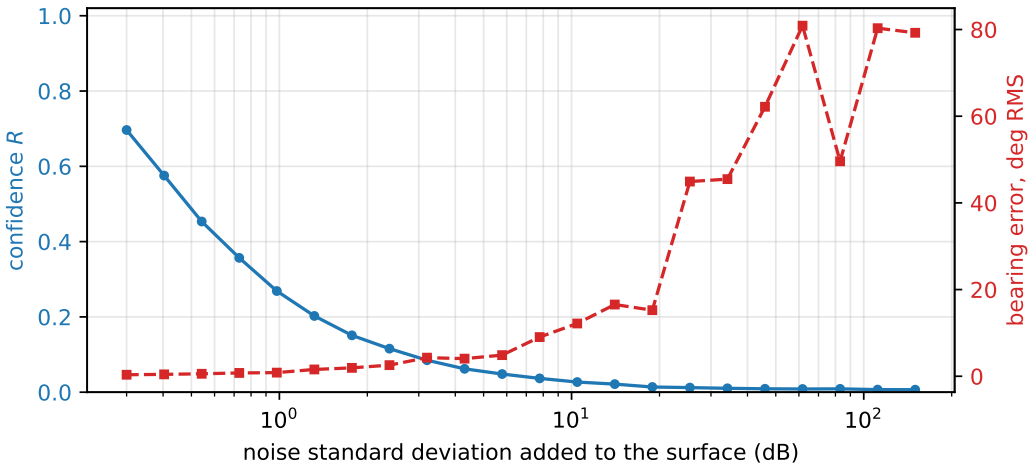


Figure 6: A clean planar surface at 45° with increasing zero-mean noise. R has already fallen below 0.2 while the bearing is still within a couple of degrees. Treat a low R as a warning to look at the plot, not as an error bar.

In practice:

- R **above about 0.4** — the field agrees over most of the area. The bearing is worth using.
- R **between 0.1 and 0.4** — partial agreement. Look at the surface before believing the number. Often the flight only sampled one side of the gradient.
- R **below 0.1** — no consensus. Usually one of the failure modes below rather than a noisy but usable estimate.

6.6 When it does not work

The method assumes the surface slopes monotonically towards the tag across the area flown. Three things break that assumption:

You flew over the tag. The surface then has an interior maximum and the gradients point inwards from all sides, cancelling — the right-hand panel of figure 5. This is a good outcome for localisation and a useless one for bearing. **Divergence** mode makes it obvious.
Terrain. Known from field experience: where there is significant topography near the aircraft the peak signal can occur slightly downhill of the tag rather than above it, and the gradient no longer points at it. Prefer the directional antenna and triangulation where terrain is a factor.
You picked a property that is not signal strength. The estimator will happily report a bearing for **Time** or **Altitude (m)**, and it will be the direction the aircraft climbed or flew, not the direction of a tag. Only SNR — and to a lesser extent STFT score — carries bearing information.

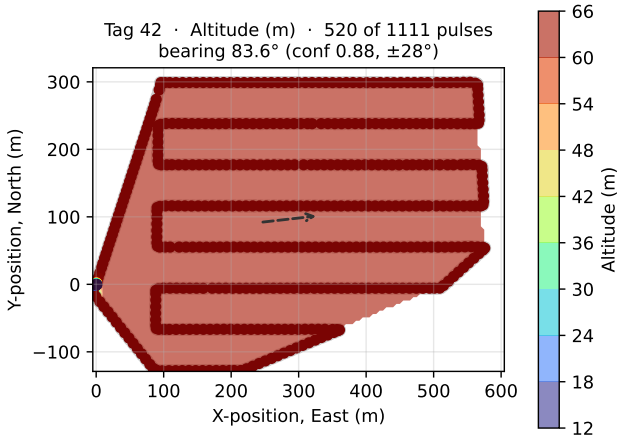


Figure 7: The third failure mode, and the reason the confidence is not a measure of correctness. This is the Day 5 flight with **Property** set to **Altitude (m)**: the surface is the aircraft's own height profile, so the field agrees strongly ($R = 0.88$, the highest number in this guide) on a bearing of 83.6° that has nothing whatever to do with the tag. A high R means the cells agree, not that you asked a sensible question.

Not yet validated against ground truth. The estimator is verified numerically — exact on clean linear fields, 45.00° on a point-source field whose true bearing is 45°, with R collapsing as noise rises — but it has not been checked against a flight with an independently known tag position. Treat bearings as indicative.

7 The exported KMZ

One archive, three folders, no network needed — every icon and image is packaged inside, so it renders correctly in the field with no connection:

- ⟨**property**⟩ **surface** — the interpolated field as a semi-transparent raster, transparent outside the area actually flown.
- **Contour lines** — level curves, switched off by default.
- **Pulses (tag N)** — one marker per pulse, coloured by the property, with the value in its balloon.

If **Plot Tag** is on, the known tag position is included as a fourth folder.

8 If something goes wrong

Symptom	Cause
Python: window opens nearly empty, no error	Tk 8.5. Rebuild the environment on a Python with Tk 8.6.
MATLAB: your edit had no effect	MATLAB cached the classdef. `clear classes` and re-run.
MATLAB: edits to the `.mlapp` keep reverting	App Designer holds its own copy and writes it back. Edit `pulseplotter2.m`.
Tag list contains a number that looks like a latitude	A rotation record was read as a pulse. Should not happen with this reader; report the log.
No surface, only dots	Fewer than four finite values, or every value identical, for the selected tag and window.
Bearing reads –	No surface, so no gradient. Widen the time or SNR window.
The plot crawls on a long flight	Grid resolution too fine. It is capped automatically, but a coarser value replots faster.

A Differences between the two implementations

The numbers are the same. The differences are all in the interface:

- The time and SNR ranges are two separate sliders each in Python, rather than one two-handled slider; Tk has no native range widget. Their bounds are printed beside them, where MATLAB has read-only Start Time and End Time fields in the control column.
- Python includes a matplotlib toolbar, so you can zoom and pan. MATLAB cannot.
- **Plot Tag** is a checkbox in Python, a slider switch in MATLAB.
- Python's export writes the KMZ directly; MATLAB goes through a temporary `.kml`.
- `MONOPOLE_SCAN_MAPPING.m`, a standalone antenna-pattern analysis script, is MATLAB-only and is not part of the plotter.

Neither has been run side by side on the same log and compared numerically. That is the obvious next check; until it is done, "same numbers" is an intention rather than a measurement.

B Reproducing the figures

Every plot in this guide is real output from the application, generated by driving the same controls a user would:

```
make -C DOCS all
```

That runs `make_figures.py` for the plots, `make_screenshots.py` for the two screenshots, and `latexmk` for the document. The two pulse logs live outside the repository, under `FLIGHT_TESTING_DATA`; pass other paths through with `make figures LOGS="day5.csv day8.csv"`.

The screenshots are macOS-only and need Screen Recording permission for whatever runs them, under **System Settings** → **Privacy & Security** → **Screen & System Audio Recording**.

The window must also fit on the main display; widen `WINDOW` in `make_screenshots.py` if yours is bigger than the 1024 px it currently asks for.

Developed under NSF awards 1556417 and 2104570. Source, licence (GPL-3.0) and working notes: `uavrt_postflight`.